

# IPC by Team THRDx

Jovita deena dsouza

## Definition

**Inter Process Communication (IPC)** is the set of methods an operating system gives running programs so they can exchange data and signal each other. Since each process lives in its own protected memory space, they can't just reach into one another, so IPC acts as the approved go-between.

## Pipes and Named Pipes

A **pipe** is a one-way channel between two processes. One side writes bytes in, the other side reads them out, and the data comes out in the same order it went in. In C you create one with `pipe()`, which hands you two file descriptors, one for reading and one for writing. Anonymous pipes usually connect a parent and child after a `fork()`, and they vanish when the processes end. The catch is that they only work between related processes and only in one direction, so two-way talk needs two pipes.

**Named pipes** (FIFOs) get around this by having a real name in the file system, created with `mkfifo()`. Any two processes can open that name and communicate, even if they were started separately and have nothing to do with each other. The pipe also stays in the file system until you delete it. Both types block automatically, so a reader waits when there's nothing to read and a writer waits when the pipe is full.

## Example

Picture a simple chat between two terminal windows on the same machine. Window one runs a program that creates a named pipe called `/tmp/chat` and waits for input. Window two runs a different program that opens `/tmp/chat` and writes a message like "hello from window two". Window one instantly picks it up and prints it. The two programs never shared variables or memory, the named pipe was the only thing connecting them.

Since this was the same template as before, I wrote a fresh version instead of repeating the last one. If you need it shorter, or with a small C snippet for `pipe()` or `mkfifo()`, I can add that.